

ASSIGNMENT QUESTIONS

1. LIST – Practical Usage Questions

Q1. Add item

Create a list called `cities = ["pokhara", "lalitpur", "hetauda"]` . Ask the user for a new city name and add it to the list. Finally, print the updated list.

Hint: Use `cities.append(new_city)` .

Q2. Remove item

Ask the user to enter a city to remove from the same list. If it exists, remove it; otherwise show "City not found".

Hint: Check with `if city in cities:` before removing.

Q3. Modify item

Change the city at index `2` to uppercase.

Hint: Index assignment $\rightarrow$ `cities[2] = cities[2].upper()` .

Q4. Insert item at specific index

Insert `"janakpur"` at index `1` in the `cities` list.

Hint: Use `insert(1, "janakpur")` .

Q5. Create a list of roll numbers and print squares

Ask the user to continuously enter roll numbers until they enter `-1` . Store them in a list `rolls` . Then print each roll number and its square.

Example output: `7  $\rightarrow$  49` , `3  $\rightarrow$  9` , etc.

Hint: While loop to collect numbers, then a for loop.

2. TUPLE – Practical Usage Questions

Q6. Show immutability

Create a tuple: `fruits = ("apple", "banana", "mango")` Try to change `"banana"` to `"BANANA"` and observe what happens. Explain the error in your own words.

Hint: Tuples cannot be modified after creation.

Q7. Convert $\rightarrow$ Modify $\rightarrow$ Convert back

Convert the tuple into a list, add `"orange"` , and convert it back to a tuple named `fruits_final` .

Hint: `temp = list(fruits)  $\rightarrow$  modify  $\rightarrow$  fruits_final = tuple(temp)` .

Q8. Indexing tuple

Create `marks = (78, 82, 91)` Print only the highest mark using indexing.

Hint: Highest is at index 2 if the tuple is sorted.

3. SET – Practical Usage Questions

Q9. Remove duplicates

Create a list: `animals = ["cat", "dog", "cow", "cat", "goat", "cow"]` Convert it into a set and print the unique animals.

Hint: Use `set(animals)`.

Q10. Set operations

Given: `team_A = {2, 4, 6, 8}` `team_B = {1, 2, 3, 4}`

Find:

- Players common in both teams
- Players only in team_A
- Players in at least one team
- Players who are in exactly one team

Hint: Use `intersection`, `difference`, `union`, `symmetric_difference`.

Q11. Intersection from lists

Given: `coding = ["alice", "bob", "dilip"]` `robotics = ["dilip", "suman", "alice"]` Find students who joined both clubs.

Hint: Convert lists to sets → intersection.

Q12. Basic set operations

Create a set: `scores = {10, 20, 30}` Add `40`. Remove `10`. Print the final set.

Hint: `scores.add()` and `scores.remove()`.

4. DICTIONARY – Practical Usage Questions

Q13. Nested dictionary

Create a dictionary called `book` with keys:

- `"title"` → `"Python Basics"`
- `"author"` → `"John Doe"`
- `"details"` → another dictionary containing:
 - `"pages"` → 250
 - `"publisher"` → `"TechPress"`

Print only the publisher name.

Hint: `book["details"]["publisher"]`.

Q14. Modify value

Change the "author" value to "Jane Smith" .

Hint: Direct reassignment: `book["author"] = "Jane Smith"` .

Q15. Add new key

Add a "year" key with value 2023 to the book dictionary.

Hint: Same as modifying: `book["year"] = 2023` .

Q16. Access deep nested values

Create:

```
car = {
    "brand": "Toyota",
    "specs": {
        "engine": {"hp": 150, "type": "petrol"},
        "wheels": 4
    }
}
```

Print only the horsepower value.

Hint: `car["specs"]["engine"]["hp"]`

Q17. Loop through dictionary

Loop through all keys and values of the car dictionary and print them.

Hint: `for key, value in car.items():`

Q18. Check existence

Check if "brand" exists in the car dictionary, and check if "Tesla" exists as a value.

Hint: Use `in car.keys()` and `in car.values()` .

🔪 BONUS QUESTION – Combination of All Topics

Q19. Student management system (mini-program)

Create a dictionary called data where each key is a student ID (e.g., 101, 102...) and the value is a **list of hobbies** they have.

Example (do NOT use this exact one):

```
data = {
    101: ["reading", "cycling"],
    102: ["gaming"],
    103: ["swimming", "cycling"],
}
```

Tasks:

1. Print all unique hobbies done by students (use sets).
2. Print IDs of students who have more than 1 hobby.
3. Print IDs of students who have "cycling" as one of their hobbies.

Hints:

- Convert all hobby lists into a set using union.
- Check length of each hobby list.

- Use 'cycling' in hobby_list .

Here are 2 more **BONUS** questions – both **practical, multi-concept**, and **different from anything shown earlier or in your class examples**.

📌 BONUS QUESTION 20 — Course Enrollment Analyzer

You are given a list of students and the subjects they take. Create the following dictionary (you may add your own sample values):

```
enroll = {
    "arjun": ["math", "english", "science"],
    "bishal": ["nepali"],
    "chandni": ["math", "computer"],
    "deepa": ["science", "math"],
    "ekta": ["computer", "english"]
}
```

Tasks:

1. Print the names of students who study **both math and science**.
2. Create a **set of all subjects** students are taking (no duplicates).
3. Create a tuple containing only the students who take **exactly 1 subject**.
4. Remove "english" from Ekta's subject list and print the updated dictionary.

Hints:

- For (1): check "math" in enroll[name] AND "science" in enroll[name] .
- For (2): Convert each list to a set and union them.
- For (3): Store names in list → convert to tuple at the end.
- For (4): Use .remove() on the list inside the dictionary.

📌 BONUS QUESTION 21 — Store Inventory System

Create a dictionary named `store` where each key is the **item name**, and value is another dictionary containing:

- "price" (integer)
- "stock" (available quantity)

Example format (use different items):

```
store = {
    "soap": {"price": 50, "stock": 25},
    "shampoo": {"price": 120, "stock": 10},
    "brush": {"price": 30, "stock": 40}
}
```

Tasks:

1. Add a new product "toothpaste" with price 90 and stock 20.
2. Reduce the stock of "soap" by 5 (simulate purchase).
3. Create a list of items that are **low stock** (stock < 15).
4. Create a set of all prices in the store (unique prices).
5. Print the price of "shampoo" .

Hints:

- Use store["toothpaste"] = { ... } to add new product.
- Reduce stock: store["soap"]["stock"] -= 5 .
- Loop through dictionary keys to check stock levels.
- Collect prices using a set.
- Access nested value: store["shampoo"]["price"] .